

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №5
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Определение характеристик
графов»

Выполнил:

студенты группы 24BBB2

Бауэр А.А.

Гусев Е.В.

Медведев Е.А.

Принял:

к.т.н. доцент Юрова О.В.

к.т.н. Деев М.В.

Пенза 2025

Название

Определение характеристик графов.

Цель работы

Научиться строить графы и матрицы к ним. Научиться определять характеристики графов.

Лабораторное задание

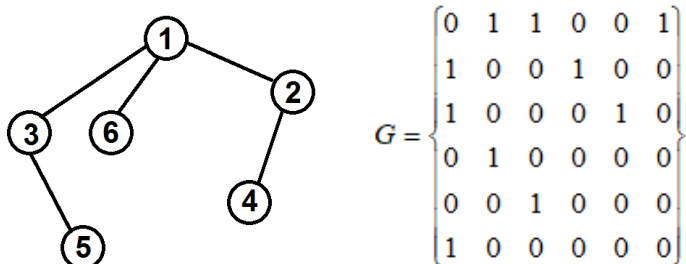


Рисунок 1 – Граф

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Определите размер графа G , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

Задание 2*

1. Постройте для графа G матрицу инцидентности.
 2. Определите размер графа G , используя матрицу инцидентности графа.
- Найдите изолированные, концевые и доминирующие вершины.

Описание метода решения задачи

Задание 1

Граф представляется в виде матрицы смежности - квадратной матрицы размером $V \times V$, где элемент $matrix[i][j] = 1$ указывает на наличие ребра между вершинами i и j . Для определения размера графа выполняется подсчет вершин (размер матрицы) и ребер (сумма элементов выше главной диагонали для неориентированного графа). Анализ специальных вершин проводится через вычисление степени каждой вершины как суммы элементов соответствующей строки матрицы: изолированные вершины имеют степень 0, концевые - степень 1, доминирующие - степень $V-1$.

Задание 2*

Матрица инцидентности строится на основе матрицы смежности. Это прямоугольная матрица размером $V \times E$, где каждый столбец соответствует ребру и содержит ровно две единицы для вершин, соединенных данным ребром. Количество ребер E определяется из матрицы смежности. Размер графа находится непосредственно из размеров матрицы инцидентности. Степени вершин

вычисляются как суммы элементов по строкам матрицы, что позволяет идентифицировать специальные вершины аналогично первому заданию.

Листинг

Файл 5LR.cpp (1 задание)

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#define V 6 //кол-во вершин

//Функция для вывода матрицы
void printMatrix(int matrix[V][V]) {
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

//Функция для определения степени вершины
int getDegree(int matrix[V][V], int vertex) {
    int degree = 0;
    for (int i = 0; i < V; i++) {
        degree += matrix[vertex][i];
    }
    return degree;
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "ru");
    //Матрица смежности
    int adjacencyMatrix[V][V] = {
        {0, 1, 1, 0, 0, 1},
        {1, 0, 0, 1, 0, 0},
        {1, 0, 0, 0, 1, 0},
        {0, 1, 0, 0, 0, 0},
        {0, 0, 1, 0, 0, 0},
        {1, 0, 0, 0, 0, 0}
    };

    printf("1. Матрица смежности графа G:\n");
    printMatrix(adjacencyMatrix);
    printf("\n");

    //Размер графа (количество вершин и ребер)
    int edges = 0;
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            edges += adjacencyMatrix[i][j];
        }
    }
}
```

```

printf("Размер графа G:\n");
printf("Количество вершин: %d\n", V);
printf("Количество ребер: %d\n", edges);
printf("\n");
//Поиск специальных вершин
printf("Специальные вершины:\n");
printf("Изолированные вершины (степень 0): ");
int hasIsolated = 0;
for (int i = 0; i < V; i++) {
    if (getDegree(adjacencyMatrix, i) == 0) {
        printf("%d ", i + 1);
        hasIsolated = 1;
    }
}
if (!hasIsolated) printf("нет");
printf("\n");
printf("Концевые вершины (степень 1): ");
int hasEnd = 0;
for (int i = 0; i < V; i++) {
    if (getDegree(adjacencyMatrix, i) == 1) {
        printf("%d ", i + 1);
        hasEnd = 1;
    }
}
if (!hasEnd) printf("нет");
printf("\n");
printf("Доминирующие вершины (степень %d): ", V - 1);
int hasDominant = 0;
for (int i = 0; i < V; i++) {
    if (getDegree(adjacencyMatrix, i) == V - 1) {
        printf("%d ", i + 1);
        hasDominant = 1;
    }
}
if (!hasDominant) printf("нет");
printf("\n");
return 0;
}

```

Файл 5LR_star.cpp (2 задание)

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#define V 6 // количество вершин
#define E 5 // количество ребер
//Функция для вывода матрицы инцидентности
void printIncidenceMatrix(int matrix[V][E]) {
    //Строки матрицы (вершины)
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < E; j++) {
            printf(" %d ", matrix[i][j]);
        }
        printf("\n");
    }
}

```

```

}

//Функция для определения степени вершины из матрицы инцидентности
int getDegreeFromIncidence(int matrix[V][E], int vertex) {
    int degree = 0;
    for (int j = 0; j < E; j++) {
        degree += matrix[vertex][j];
    }
    return degree;
}

int main() {
    setlocale(LC_ALL, "ru");
    //Исходная матрица смежности для построения матрицы
инцидентности
    int adjacencyMatrix[V][V] = {
        {0, 1, 1, 0, 0, 1},
        {1, 0, 0, 1, 0, 0},
        {1, 0, 0, 0, 1, 0},
        {0, 1, 0, 0, 0, 0},
        {0, 0, 1, 0, 0, 0},
        {1, 0, 0, 0, 0, 0}
    };

    //Построение матрицы инцидентности
    int incidenceMatrix[V][E];
    //Инициализация матрицы инцидентности нулями
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < E; j++) {
            incidenceMatrix[i][j] = 0;
        }
    }
    //Заполнение матрицы инцидентности
    int edgeCount = 0;
    for (int i = 0; i < V; i++) {
        for (int j = i + 1; j < V; j++) {
            if (adjacencyMatrix[i][j] == 1) {
                incidenceMatrix[i][edgeCount] = 1;
                incidenceMatrix[j][edgeCount] = 1;
                edgeCount++;
            }
        }
    }
    printf("Матрица инцидентности графа G:\n");
    printIncidenceMatrix(incidenceMatrix);
    printf("\n");

    // 2. Размер графа через матрицу инцидентности
    printf("Размер графа G:\n");
    printf("Количество вершин: %d\n", V);
    printf("Количество ребер: %d\n", E);
    printf("\n");

    //Специальные вершины через матрицу инцидентности
    printf("Специальные вершины:\n");
    printf("Изолированные вершины (степень 0): ");

```

```

int hasIsolated = 0;
for (int i = 0; i < V; i++) {
    if (getDegreeFromIncidence(incidenceMatrix, i) == 0) {
        printf("v%d ", i + 1);
        hasIsolated = 1;
    }
}
if (!hasIsolated) printf("нет");
printf("\n");
printf("Концевые вершины (степень 1): ");
int hasEnd = 0;
for (int i = 0; i < V; i++) {
    if (getDegreeFromIncidence(incidenceMatrix, i) == 1) {
        printf("%d ", i + 1);
        hasEnd = 1;
    }
}
if (!hasEnd) printf("нет");
printf("\n");
printf("Доминирующие вершины (степень %d): ", V - 1);
int hasDominant = 0;
for (int i = 0; i < V; i++) {
    if (getDegreeFromIncidence(incidenceMatrix, i) == V - 1) {
        printf("%d ", i + 1);
        hasDominant = 1;
    }
}
if (!hasDominant) printf("нет");
printf("\n");
return 0;
}

```

Текст к программе

1. 5LR.cpp (1 задание)

Программа реализует анализ неориентированного графа с использованием двух матричных представлений: матрицы смежности и матрицы инцидентности. В программе задан конкретный граф из 6 вершин, который описывается матрицей смежности, где каждый элемент показывает наличие или отсутствие связи между соответствующими вершинами. Для задания 1 программа на основе матрицы смежности определяет размер графа (количество вершин и рёбер) и классифицирует вершины на изолированные, концевые и доминирующие путём вычисления степени каждой вершины как суммы элементов её строки в матрице.

2. 5LR_star.cpp (2 задание)

Программа преобразует матрицу смежности в матрицу инцидентности, где столбцы соответствуют рёбрам графа, а строки - вершинам, причём каждый столбец содержит ровно две единицы для вершин, соединённых данным ребром. Анализ специальных вершин повторяется с использованием матрицы инцидентности, где степень вершины вычисляется как сумма элементов её строки.

Результаты работы программы

Результаты работы программы показаны на рисунках 2-3.

I. 5LR.cpp (1 задание)

```

Матрица смежности графа G:
0 1 1 0 0 1
1 0 0 1 0 0
1 0 0 0 1 0
0 1 0 0 0 0
0 0 1 0 0 0
1 0 0 0 0 0

Размер графа G:
Количество вершин: 6
Количество ребер: 5

Специальные вершины:
Изолированные вершины (степень 0): нет
Концевые вершины (степень 1): 4 5 6
Доминирующие вершины (степень 5): нет

```

Рисунок 2 – Корректная работа программы

II. 5LR_star.cpp (2 задание)

```

Матрица инцидентности графа G:
1 1 1 0 0
1 0 0 1 0
0 1 0 0 1
0 0 0 1 0
0 0 0 0 1
0 0 1 0 0

Размер графа G:
Количество вершин: 6
Количество ребер: 5

Специальные вершины:
Изолированные вершины (степень 0): нет
Концевые вершины (степень 1): 4 5 6
Доминирующие вершины (степень 5): нет

```

Рисунок 3 – Корректная работа программы

Выводы

Научились определять характеристики графов. Выяснили, что оба задания демонстрируют эквивалентность матричных представлений для анализа свойств графа, подтверждая, что различные математические модели могут быть использованы для решения одних и тех же задач теории графов с одинаковыми результатами. Программы наглядно показывают взаимосвязь между разными способами представления графов и их практическое применение для анализа структурных свойств сетей.